27.04.2026

MARMARA ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

8086 ASSEMBLY İLE LABİRENTTEN KAÇIŞ OYUNU

PROJE RAPORU

GitHub Link

Ders: BLM2008.2-Mikroişlemciler

Hazırlayan: Kerem Sadık HACIÖMEROĞLU

170424062

İÇİNDEKİLER

1. PROJE ÖZETİ

2. OYUNUN TEMEL ÖZELLİKLERİ VE KONTROLLER

3. KOD YAPISI VE TEKNİK ANALİZ

- 3.1. Veri Tanımlamaları (.data)
- 3.2. Grafik ve Ekran Yönetimi
- 3.3. Hareket Mekanizması ve Çarpışma Testi
- 3.4. Dinamik Zamanlayıcı (Timer)

4. OYUN AKIŞI (GÖRSEL ANALİZ)

- 4.1. Başlangıç Ekranı
- 4.2. Oynanış ve Seviye Tasarımları
- 4.3. Seviye Geçiş Mekanizması
- 4.4. Final Labirenti (Büyük Usta Seviyesi)
- 4.5. Oyun Sonu Senaryoları

5. FONKSİYONEL ALT PROGRAMLAR

- 5.1. print_timer: Sayısal Verilerin ASCII Dönüşümü
- 5.2. put_char: Karakter ve Renk Render İşlemi

6. DERİNLEMESİNE TEKNİK KOD ANALİZİ

7. SONUÇ

8. KAYNAKÇA

1. Proje Özeti

Bu proje; 8086 Assembly dili kullanılarak yapılan,zaman karşı yarışılan, 3 farklı zorluk seviyesinden oluşan bir labirent oyunudur. Oyun, metin tabanlı grafik modunda (`INT 10h`) karakterleri manipüle ederek görsel bir arayüz sunar. Oyuncunun amacı, duvarlara çarpmadan sınırlı bir süre içinde çıkış kapısına ulaşmaktır.

2. Oyunun Temel Özellikleri ve Kontroller

Seviye Sistemi: Toplam 3 seviye (Lvl 1, Lvl 2, Lvl 3 - Büyük Usta Labirenti).

Zamanlayıcı: Seviye 1 için 60 saniye, diğerleri için 90 saniye süre sınırı.

Kontroller: Yön tuşları (Yukarı, Aşağı, Sol, Sağ).

Görsel Karakterler: Oyuncu: Elmas özel karakter (ASCII 4).

Duvar: Dolu blok (ASCII 219).

Çıkış: Taralı blok (ASCII 178).

3. Kod Yapısı ve Teknik Analiz

3.1. Veri Tanımlamaları (.data)

Kodun bu kısmında oyunun "haritaları" metin dizileri şeklinde saklanır. # karakterleri duvarları, P oyuncu başlangıcını, E ise çıkışı temsil eder. Burada her labirent 20x20'lik bir matris olarak kurgulanmıştır. Program, `level_ptr` değişkeni sayesinde o an hangi seviyede olduğunuzu takip eder.

```
lvl1 db "####################"
     db "#P#        #       #"
     db "# # ##### # ###### #"
     db "#   #   #     #    #"
     db "##### # # ###### # #"
     db "#     # # #      # #"
     db "# ##### # # ###### #"
     db "# #     # #        #"
     db "# # ##### ##########"
     db "# #                #"
     db "# # ##############"
     db "# # #            # #"
     db "# # # ########## # #"
     db "# # # #        # # #"
     db "# # # # ###### # # #"
     db "#   # # #    # # # #"
     db "##### # # ## # # # #"
     db "#       #    #     #"
     db "##################E#"
     db "####################", 0
```

```
lvl2 db "####################"
     db "#P#    #       #   #"
     db "# # # # ### # #### #"
     db "# # #   #   #    # #"
     db "# # ##### ###### # #"
     db "#   #   #      # # #"
     db "### # # ###### # # #"
     db "#   # #      # #   #"
     db "# ### ###### # #####"
     db "#   #      # #     #"
     db "# # ###### # #### #"
     db "# #      # #   #   #"
     db "# ###### # ### # ###"
     db "#      # #     #   #"
     db "###### # ##### ### #"
     db "#    # #      # #  #"
     db "# ## # ##### # # ###"
     db "#    #      #     #"
     db "######### ########E#"
     db "####################", 0
lvl3 db "####################"
     db "#P#    #       #   #"
     db "# # # ### ### # ## #"
     db "# # #   #   # #   #"
     db "# # ### ### # #### #"
     db "#   #   #   #    # #"
     db "### # ### ###### # #"
     db "#   #   #      # # #"
     db "# ### ####### ## # #"
     db "# #   #     #  # # #"
     db "# # ### ### ###### #"
     db "# #   #   #      # #"
     db "# ### ### ###### # #"
     db "#   #   #   #    # #"
     db "### ### ### # #### #"
     db "#     #     #    # #"
     db "# ##### ######## # #"
     db "#     #        #   #"
     db "# ########## #### E#"
     db "####################", 0
```

3.2. Grafik ve Ekran Yönetimi

Oyun, DOS kesmelerini (`interrupts`) kullanarak ekranı yönetir.

`clear_screen`: Ekranı temizlemek için `INT 10h / AH=00h` kullanarak video modunu 80x25 metin moduna (mode 03h) ayarlar.

`put_char`: Belirli bir koordinata karakter basar. Burada dikkat çeken nokta, ASCII kodlarının görselliği artırmak için değiştirilmesidir:

`WALL_CHAR` (#) ekrana basılırken `219` (Tam blok) koduna dönüştürülür.

`PLAYER_CHAR` (P) ekrana basılırken `4` (Elmas sembolü) koduna dönüştürülür.

3.3. Hareket Mekanizması ve Çarpışma Testi

Kullanıcı bir tuşa bastığında **`validate`** etiketi altında şu işlemler yapılır:

1. Oyuncunun gitmek istediği yeni koordinat hesaplanır.
2. Bu koordinattaki karakter kontrol edilir.
3. Eğer karakter # ise hareket iptal edilir.
4. Eğer karakter E ise `level_up` fonksiyonu çağrılır.
5. Eğer boşluk ise oyuncunun eski yeri temizlenir ve yeni yerine çizilir.

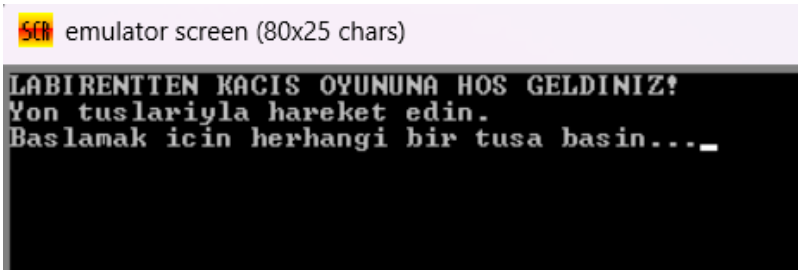
3.4. Dinamik Zamanlayıcı (Timer)

Zamanlayıcı, BIOS'un saat kesmesini (`INT 21h / AH=2Ch`) kullanır. DH kaydı üzerinden saniye bilgisi alınır ve her saniye değişiminde `seconds` değişkeni bir azaltılır.

```
update_timer:
   mov ah, 2Ch
   int 21h        ; Sistem saatini oku
   mov al, dh      ; DH = Saniye
   cmp al, prev_second
   je timer_done   ; Saniye henüz değişmediyse çık
  dec seconds     ; Değiştiyse süreyi azalt
```

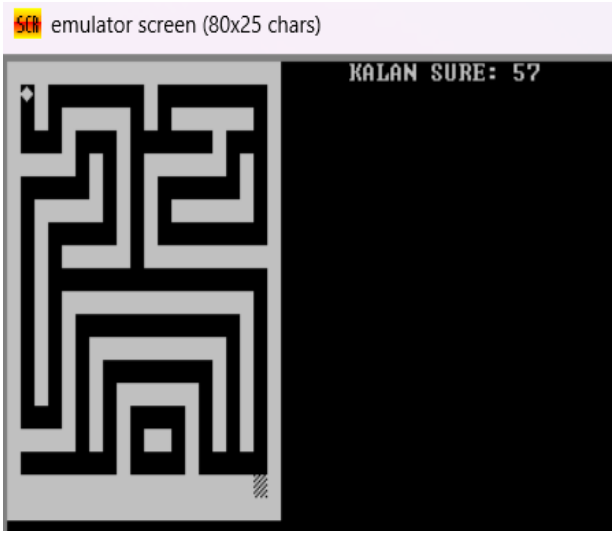
4. Oyun Akışı (Görsel Analiz)

4.1.Başlangıç: `msg_welcome` metni ekrana gelir.

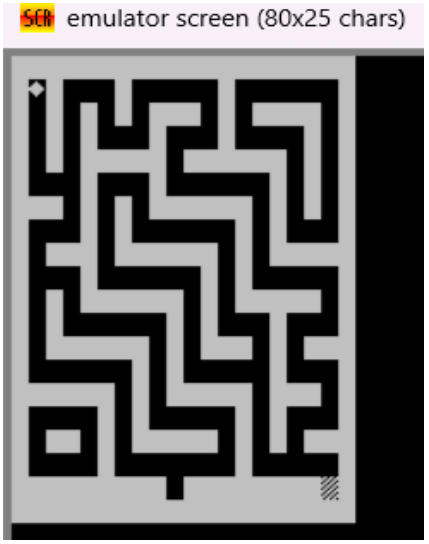


4.2.Oynanış: Labirent çizilir ve sağ üst köşede "KALAN SURE" dinamik olarak güncellenir.

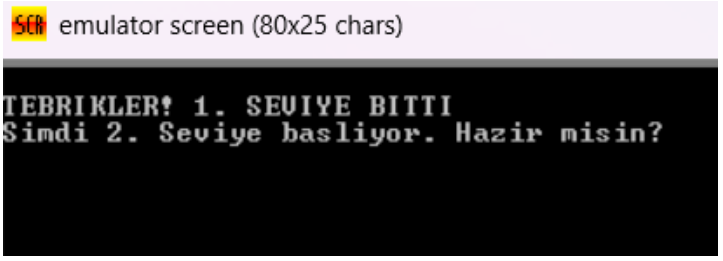
1.SEVİYE

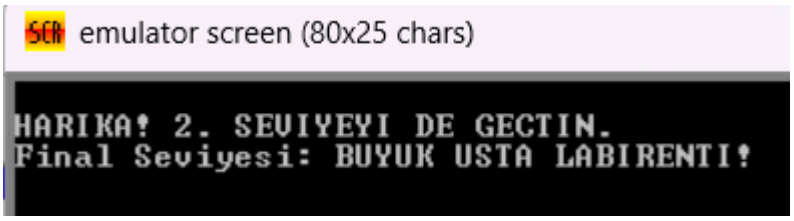


2.SEVİYE

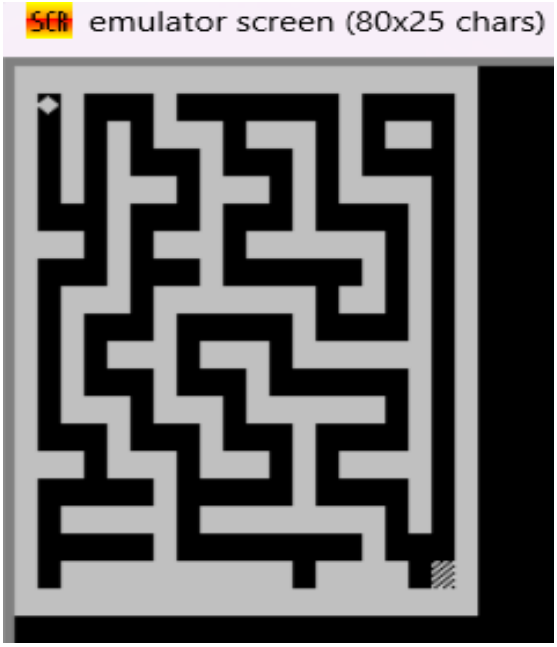


4.3.Seviye Geçişi: Çıkışa ulaşıldığında bir sonraki seviye mesajı gelir.



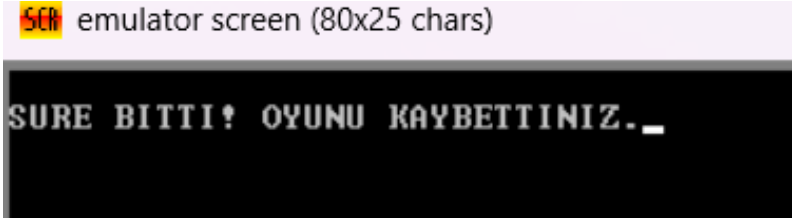


4.4.Final Labirenti: Seviye 3, "Büyük Usta Labirenti" adıyla daha karmaşık bir yolla karşımıza çıkar.

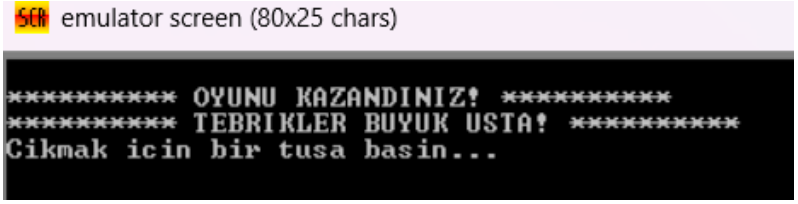


4.5.Oyun Sonu Senaryoları:

Süre biterse "SURE BITTI" mesajı görüntülenir.



Oyun başarıyla tamamlanırsa ekranda final mesajı görüntülenir.



6. Derinlemesine Kod Analizi

1-Karakter Sabitleri

```
WALL_CHAR    equ '#'

PLAYER_CHAR  equ 'P'

EXIT_CHAR    equ 'E'

SPACE_CHAR   equ ' '
```

equ (equate) ile sembolik isimlendirme yapılır. Derleme anında # karakteri WALL_CHAR ile değiştirilir. Bu, kod okunabilirliğini artırır ve değişiklik yapmayı kolaylaştırır

2-Klavye Scan Kodları

```
KEY_UP       equ 48h
KEY_DOWN     equ 50h
KEY_LEFT     equ 4Bh
KEY_RIGHT    equ 4Dh
```

BIOS int 16h çağrısından dönen AH register'ındaki scan code değerleridir. Yön tuşlarının ASCII karşılığı olmadığı için bu değerlerle karşılaştırma yapılır.

3-Oyun Durum Değişkenleri

```
cur_x dw 1
cur_y dw 1
level_num db 1
level_ptr dw offset lvl1
```

cur_x, cur_y : Oyuncunun labirent üzerindeki sütun (X) ve satır (Y) koordinatları (1..18 arası).

level_num : Şu anki seviye numarası (1, 2 veya 3).

level_ptr : Şu anki seviyenin harita verisinin başlangıç adresini (offset) tutan işaretçi (pointer). Seviye geçişlerinde bu pointer güncellenir.

4-Zamanlayıcı (Timer) Değişkenleri

```
timer_started db 0
seconds db 60
prev_second db 0
```

timer_started : 0 = süre sayacı henüz başlamadı (ilk tuşa basılması bekleniyor), 1 = başladı.

seconds : Kalan süre (saniye cinsinden). Seviye 1 için 60, seviye 2 ve 3 için 90.

prev_second : Sistem saatinden okunan bir önceki saniye değeri. Bu değer değiştiğinde seconds azaltılır.

5-Mesaj (String) Değişkenleri

msg_timer db "KALAN SURE: $"
msg_gameover db 0dh, 0ah, "SURE BITTI! OYUNU KAYBETTINIZ.$"
msg_welcome db "LABIRENTTEN KACIS OYUNUNA HOS GELDINIZ!", 0dh, 0ah ...

Tüm mesajlar dolar işareti ($) ile sonlandırılmıştır, çünkü DOS int 21h / ah=09h fonksiyonu bir string'i $ görene kadar ekrana yazdırır. 0dh, 0ah (CR, LF) satır başı yapar.

6-Program Girişi (main)

```
main:
mov ax, @data
mov ds, ax
call clear_screen
lea dx, msg_welcome
mov ah, 09h
int 21h
mov ah, 00h
int 16h
```

@data segmentinin adresi DS'ye yüklenir. Ekran temizlenir, hoş geldin mesajı basılır. int 16h / ah=00h herhangi bir tuşa basılmasını bekler (bloklayıcı).

7-Seviye Başlatma (init_game)

```
init_game:
cmp level_num, 1
je set_60
cmp level_num, 2
je set_90
mov seconds, 90 ; level_num 3 ise buraya gelir
jmp continue_init
set_60:
mov seconds, 60
jmp continue_init
set_90:
mov seconds, 90
continue_init:
mov timer_started, 0
mov prev_second, 0
call clear_screen
call draw_map
```

Seviyeye göre seconds değişkeni ayarlanır. Seviye 3 için ayrı bir cmp yapılmaz, çünkü 1 ve 2 değilse otomatik olarak 3'tür. Timer sıfırlanır, ekran temizlenir, harita çizilir.

8-Ana Oyun Döngüsü (game_loop)

```
game_loop:
mov dh, byte ptr [cur_y]
mov dl, byte ptr [cur_x]
mov al, PLAYER_CHAR
call put_char
cmp timer_started, 1
jne check_input
```

```
call update_timer
check_input:
mov ah, 01h
int 16h
jz game_loop
mov ah, 00h
int 16h
cmp timer_started, 0
jne already_started
mov timer_started, 1
mov ah, 2Ch
int 21h
mov prev_second, dh
call print_timer
already_started:
cmp al, 27
je exit
mov cx, [cur_x]
mov dx, [cur_y]
cmp ah, KEY_UP ...
```

Oyuncu her döngüde ekrana yeniden çizilir. int 16h / ah=01h ile tuş kontrolü non-blocking yapılır; böylece süre sürekli akar. İlk tuşta timer başlatılır. ESC tuşu ile çıkış, yön tuşları ile validate'e atlanır.

9-Hareket Doğrulama (validate)

```
validate:
push bx
push si
mov ax, dx ; AX = Y
```

```
mov bl, 20
mul bl ; AX = Y * 20
add ax, cx ; AX = Y*20 + X
mov si, ax
add si, level_ptr ; SI = harita başı + indeks
mov al, [si]
pop si
pop bx
cmp al, WALL_CHAR
je game_loop
cmp al, EXIT_CHAR
je level_up
push cx
push dx
mov dh, byte ptr [cur_y]
mov dl, byte ptr [cur_x]
mov al, SPACE_CHAR
call put_char

pop dx
pop cx
mov [cur_x], cx
mov [cur_y], dx
jmp game_loop
```

Koordinat-bellek dönüşümü : (Y*20) + X formülü ile harita içindeki ofset hesaplanır.

level_ptr sayesinde doğru seviyenin haritasına erişilir.

Duvar kontrolü, çıkış kontrolü yapılır. Uygun hareketse eski konum temizlenir, yeni konum kaydedilir.

10-Zamanlayıcı Güncelleme (update_timer)

```
update_timer:
push ax bx cx dx
mov ah, 2Ch
int 21h
mov al, dh
cmp al, prev_second
je timer_done
mov prev_second, al
dec seconds
call print_timer
cmp seconds, 0
jle game_over
timer_done:
pop dx cx bx ax
ret
```

int 21h / ah=2Ch ile sistem saatinden saniye (DH) alınır.

Saniye değiştiğinde seconds azaltılır ve ekran güncellenir.

seconds <= 0 ise game_over çağrılır.

11-Süre Yazdırma (print_timer)

```
print_timer:
push ax bx cx dx
mov ah, 02h
mov bh, 0
mov dh, 0
mov dl, 25
int 10h
```

```
lea dx, msg_timer
mov ah, 09h
int 21h
mov al, seconds
xor ah, ah
mov bl, 10
div bl
mov cx, ax
mov dl, cl
add dl, '0'
mov ah, 02h
int 21h
mov dl, ch
add dl, '0'
int 21h
pop dx cx bx ax
ret
```

İmleç (0,25) konumuna taşınır, "KALAN SURE: " yazılır.

seconds değeri div bl ile onlar ve birler basamağına ayrılır; her basamak ASCII'ye çevrilip ekrana basılır.

12-Seviye Atlama (level_up)

```
level_up:
inc level_num
call clear_screen
cmp level_num, 2
je show_lvl2_msg
cmp level_num, 3
```

```
je show_lvl3_msg
jmp win_final

show_lvl2_msg:
lea dx, msg_lvl2
mov ah, 09h
int 21h
mov ah, 0Ch
mov al, 0
int 21h
mov ah, 00h
int 16h
mov level_ptr, offset lvl2
mov cur_x, 1
mov cur_y, 1
jmp init_game
```

Seviye numarası artırılır, ilgili mesaj gösterilir, klavye tamponu temizlenir, tuş beklenir.

level_ptr yeni seviyenin adresine yönlendirilir, oyuncu koordinatları sıfırlanır ve init_game ile yeni seviye başlatılır.

13-Oyun Bitti ve Kazanma

```
game_over:
pop dx cx bx ax
call clear_screen
lea dx, msg_gameover
mov ah, 09h
int 21h
mov ah, 00h
```

```
int 16h
jmp exit

win_final:
lea dx, msg_win
mov ah, 09h
int 21h
mov ah, 0Ch
mov al, 0
int 21h
mov ah, 00h
int 16h
jmp exit
```

Her iki durumda da ilgili mesaj gösterilir, tuş beklenir ve program sonlandırılır.

14-Çizim ve Grafik Prosedürleri

```
draw_map:
mov si, level_ptr
mov dh, 0
dr_row:
mov dl, 0
dr_col:
mov al, [si]
cmp al, 0
je dr_done
call put_char
inc si
inc dl
cmp dl, 20
jne dr_col
```

```
inc dh
cmp dh, 20
jne dr_row
dr_done:
ret
```

İç içe iki döngü ile 20x20 harita taranır. Her karakter put_char ile ekrana basılır.

15-put_char

```
put_char:
push ax bx cx dx
push ax
mov ah, 02h
mov bh, 0
int 10h
pop ax
cmp al, WALL_CHAR
jne check_player
mov al, 219
jmp do_print
check_player:
cmp al, PLAYER_CHAR
jne check_exit
mov al, 4
jmp do_print
check_exit:
cmp al, EXIT_CHAR
jne do_print
mov al, 178
do_print:
```

```
mov ah, 09h
mov bh, 0
mov bl, 07h
mov cx, 1
int 10h
pop dx cx bx ax
ret
```

İmleç (DH,DL) konumuna taşınır. ASCII dönüşümü : # → 219 (█), P → 4 (♦), E → 178 (▒). int 10h / ah=09h ile karakter belirtilen renkte (07h = gri) basılır.

16-clear_screen

```
clear_screen:
mov ax, 0003h
int 10h
ret
```

ax=0003h ile 80x25 metin modu (renkli) ayarlanır ve ekran temizlenir.

7. SONUÇ

Donanım ve Yazılım Etkileşimi: İşlemci yazmaçlarının (registers) etkin kullanımı ve BIOS/DOS kesmeleri (interrupts) aracılığıyla donanım birimlerinin (ekran, klavye, sistem saati) doğrudan nasıl kontrol edildiği deneyimlenmiştir.

Algoritmik Verimlilik: Kısıtlı bellek ve işlem kapasitesine sahip bir ortamda, labirent haritalama ve koordinat-indeks dönüşümü gibi matematiksel modellerin önemi kavranmıştır.

Hata Yönetimi ve Optimizasyon: Assembly dilinin doğası gereği, bellek adresleme hataları ve mantıksal döngülerin (infinite loops) ayıklanması noktasında disiplinli bir programlama yaklaşımı sergilenmiştir.

Kullanıcı Deneyimi: Zamanlayıcı (timer) ve artan zorluk seviyeleriyle, teknik bir çalışmanın nasıl eğlenceli bir son kullanıcı ürününe dönüştürülebileceği gösterilmiştir.

8. KAYNAKÇA

1.Intel 8086 Instruction Set Reference: Assembly komutlarının (MOV, CMP, JMP, MUL vb.) çalışma prensipleri ve bayraklar (flags) üzerindeki etkileri.

2.ASCII Character Codes (Extended Set): Labirent duvarları, oyuncu ikonu ve çıkış kapısı için kullanılan genişletilmiş ASCII karakter tablosu.

3.Ralf Brown's Interrupt List: `INT 10h` (Video Hizmetleri), `INT 16h` (Klavye Hizmetleri) ve `INT 21h` (DOS Fonksiyonları) kapsamlı teknik dokümantasyonu.